



SCHOOL OF COMPUTING

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the Cornerstone project work entitled

“Software Management Database Application”

is submitted by

VENNA GANESH MANIKANTA (25B11AIC74)

POLABATTULA MAHESH (25B11AI392)

BATTU BHANU KUMAR (25B11AI106)

KUNDRAPU KURMANADH (25B11AI614)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2026-2027.

Project Guide

Mrs. G L Narasamba Vanguri,
M.Tech.,(Ph.D),
Assistant Professor,
Department of AI & ML.

Head of the Department

Dr. Kovvuri N Bhargavi, Ph.D,
Associate Professor,
Department of AI & ML.

DECLARATION

We hereby declare that the Cornerstone project entitled “**Software Academic Management Database Application**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Artificial Intelligence and Machine Learning**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

V GANESH MANIKANTA (25B11AIC74)

P MAHESH (25B11AI392)

B BHANU KUMAR (25B11AI106)

K KURMANADH (25B11AI614)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. G L Narasamba Vanguri , Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. R Suneetha, Assistant Professor, Dr. B. Manikyala Rao, Assistant Professor and Dr. D Veeraswamy, Assistant Professor**, Department of Artificial Intelligence and Machine Learning, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Kovvuri N Bhargavi, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

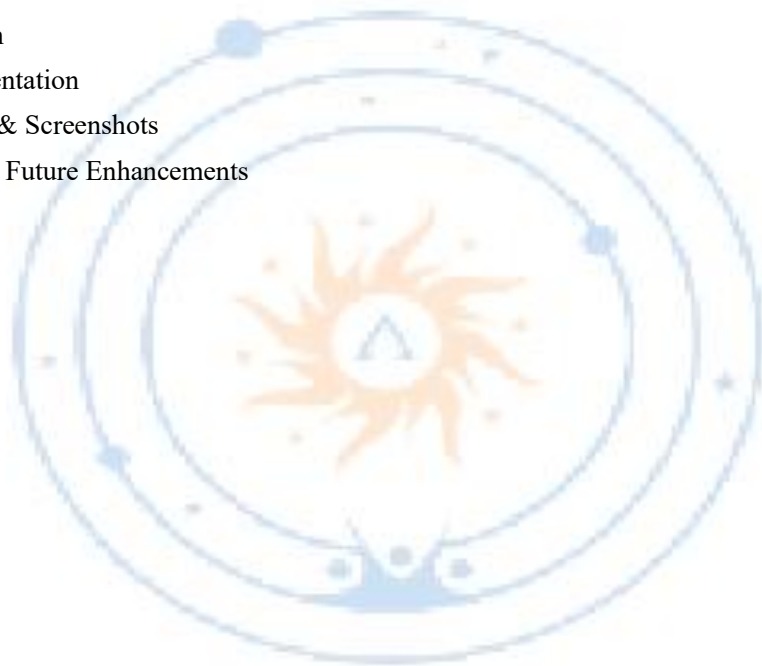
We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

1. Cover Page
2. Certificate
3. Acknowledgement
4. Table of Contents
5. Abstract
6. Introduction
7. System Analysis
8. Database Design
9. ER Diagram
10. Relational Schema
11. Table Design
12. Normalization
13. SQL Implementation
14. SQL Queries & Screenshots
15. Conclusion & Future Enhancements
16. References



ABSTRACT

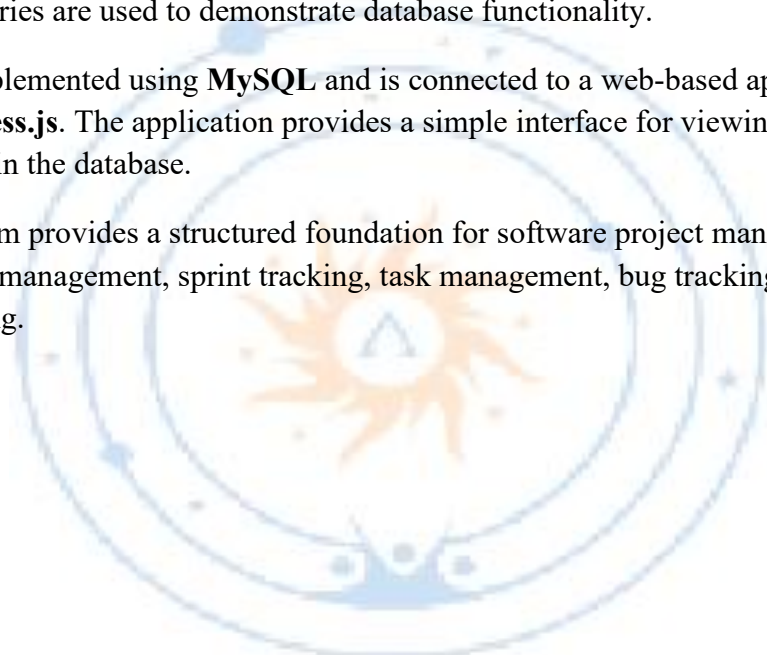
The **Software Management Database Application** is designed to manage software development activities through a centralized relational database. The system stores and manages information related to employees, projects, teams, team members, sprints, tasks, bugs, and clients.

The database provides a structured way to maintain software project information and establish relationships between different entities involved in the software development process. It helps reduce data redundancy, maintain data integrity through primary and foreign keys, and support efficient retrieval of information using SQL queries.

The system allows project-related information such as project managers, development teams, sprint details, assigned tasks, reported bugs, and client information to be stored and managed systematically. SQL operations including data insertion, updating, deletion, filtering, joins, aggregation, grouping, sorting, and subqueries are used to demonstrate database functionality.

The database is implemented using **MySQL** and is connected to a web-based application using **Node.js and Express.js**. The application provides a simple interface for viewing and managing information stored in the database.

The proposed system provides a structured foundation for software project management, employee management, team management, sprint tracking, task management, bug tracking, client management, and project reporting.



LIST OF FIGURES

Figure 1. ER Diagram of Software Management Database

Figure 2. Three-Layer Architecture of Software Management Application

Figure 3. Software Management System Dashboard

Figure 4. Employee Management Screen

Figure 5. Project Management Screen

Figure 6. Team Management Screen

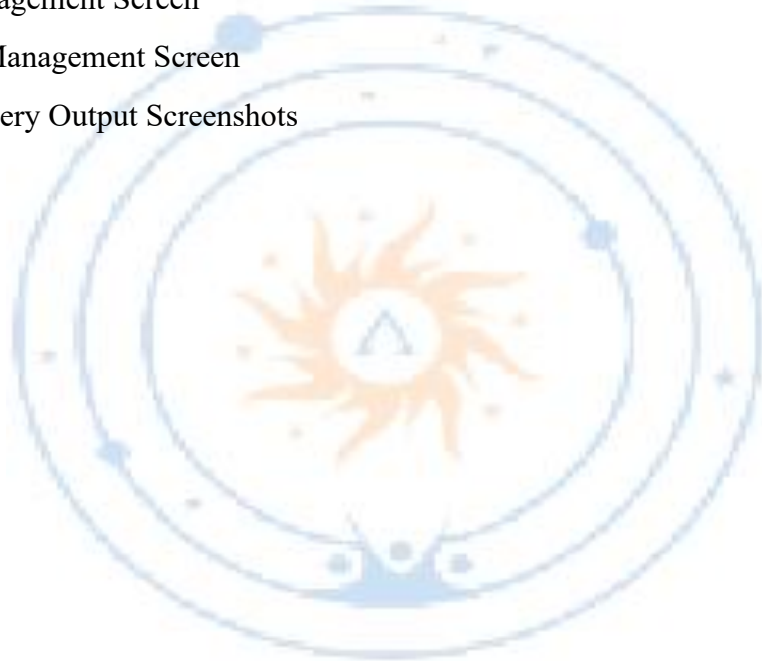
Figure 7. Sprint Management Screen

Figure 8. Task Management Screen

Figure 9. Bug Management Screen

Figure 10. Client Management Screen

Figure 11. SQL Query Output Screenshots



LIST OF TABLES

Table 1. Major Database Entities

Table 2. Employee Table Structure

Table 3. Project Table Structure

Table 4. Team Table Structure

Table 5. Team_Member Table Structure

Table 6. Sprint Table Structure

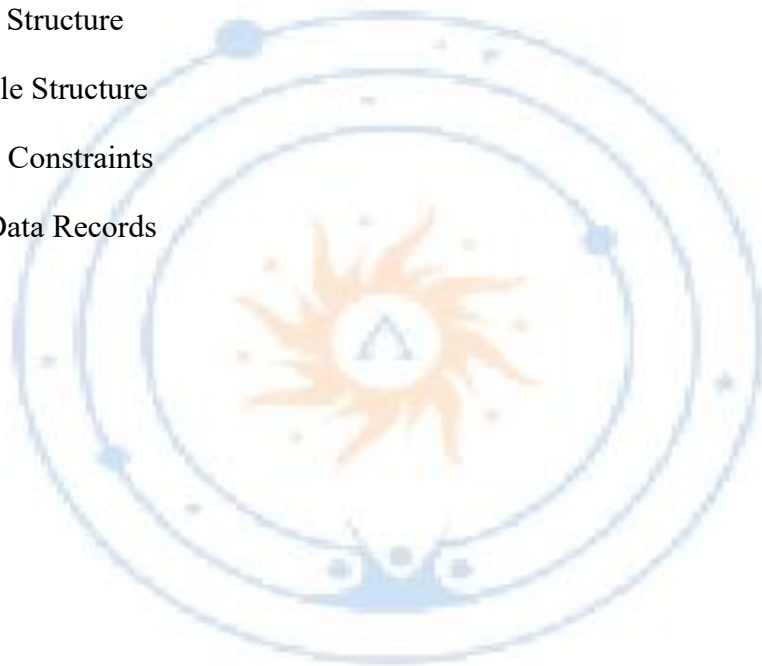
Table 7. Task Table Structure

Table 8. Bug Table Structure

Table 9. Client Table Structure

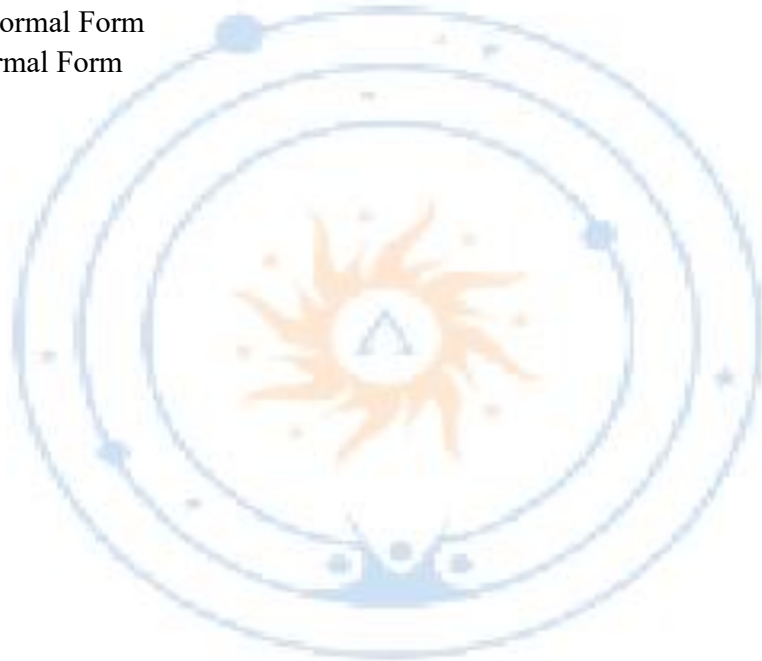
Table 10. Keys and Constraints

Table 11. Sample Data Records



ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- Crud -Create, read, Update, Delete
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form



1. INTRODUCTION

1.1 Brief Information about the Project

The **Software Management Database Application** is a relational database system designed to organize and manage information related to software development projects. The system provides centralized storage for employee, project, team, sprint, task, bug, and client information.

The database establishes relationships between these entities using primary keys and foreign keys. It enables efficient storage, retrieval, and management of software project data using SQL.

1.2 Motivation and Contribution of Project

Software development projects involve large amounts of related information, including employees, projects, development teams, tasks, sprints, bugs, and clients. Managing this information using separate files or spreadsheets can lead to duplication, inconsistency, and difficulty in retrieving related information.

The project provides a centralized relational database to organize this information and maintain relationships between different entities.

The major contributions of the project are:

- Centralizing software project information in a relational database.
- Maintaining employee and project records systematically.
- Managing teams and team members.
- Tracking sprints and assigned tasks.
- Recording and managing software bugs.
- Maintaining client information.
- Reducing data redundancy and inconsistency.
- Maintaining referential integrity using primary and foreign keys.
- Supporting efficient SQL queries and reporting.

1.3 Objectives of the Project

- Develop a centralized database for software project management.
- Maintain employee information.
- Store and manage project details.
- Manage teams and team members.
- Maintain sprint information.
- Track tasks assigned to employees.

- Record and manage software bugs.
- Maintain client information.
- Establish relationships between related entities.
- Apply database constraints to maintain data integrity.
- Perform efficient SQL queries for data retrieval and reporting.

1.4 Scope of the Project

The scope of the project covers the database-oriented management of software development information. The core database includes **Employee, Project, Team, Team_Member, Sprint, Task, Bug, and Client** entities.

The database can be used to store, retrieve, update, and analyze project-related information. It can also be extended in the future with additional modules such as time tracking, project cost management, resource allocation, notifications, and advanced reporting.



2. SYSTEM ANALYSIS

2.1 Existing System

In a software development environment, information related to employees, projects, teams, tasks, sprints, bugs, and clients may be maintained using separate files, spreadsheets, or manually managed records. Such an approach can make it difficult to maintain consistent and centralized information.

The existing approach may result in:

- Duplication of project and employee information.
- Difficulty in maintaining relationships between different records.
- Increased possibility of data inconsistency.
- Difficulty in tracking tasks, sprints, and bugs.
- Time-consuming retrieval of required information.
- Limited ability to generate SQL-based reports and summaries.
- Difficulty in maintaining data integrity between related records

2.2 Proposed System

The proposed **Software Management Database Application** provides a centralized relational database for managing information related to software development activities.

The system stores and organizes information about Employees ,Projects ,Teams ,Team members ,Sprints ,Tasks ,Bugs ,Clients

Primary keys and foreign keys are used to establish relationships between entities and maintain referential integrity. SQL queries are used to retrieve, filter, join, group, and analyze the stored information.

The proposed database provides a structured approach for managing software development data while reducing redundancy and improving consistency.

2.3 Functional Requirements

The database system should support the following functions:

1. Store and manage employee information.
2. Store project details and associate projects with managers.
3. Create and manage teams for projects.
4. Associate employees with teams.

5. Store sprint details for projects.
6. Assign tasks to employees and sprints.
7. Record and manage software bugs.
8. Store client information associated with projects.
9. Retrieve information using SQL queries.
10. Perform joins between related tables.
11. Perform aggregate operations such as COUNT, SUM, AVG, MIN, and MAX.
12. Filter and group records using SQL clauses.
13. Maintain data integrity using primary-key and foreign-key constraints.

2.4 Non-Functional Requirements

The database should satisfy the following requirements:

- **Data Integrity:** Related records should maintain valid relationships through constraints.
- **Consistency:** The same information should be maintained consistently across related records.
- **Reliability:** Database operations should produce accurate and dependable results.
- **Maintainability:** The database structure should be organized so that tables and relationships can be maintained easily.
- **Scalability:** The database should allow additional employees, projects, teams, tasks, bugs, and clients to be added in the future.
- **Security:** Database credentials and other sensitive configuration information should be protected.
- **Usability:** SQL queries should allow users to retrieve required information efficiently.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.

3. DATABASE DESIGN

3.1 Overview

The Software Management Database Application uses a relational database model to organize information related to software development activities. The database is designed using multiple related tables, where each table represents a specific entity.

The database contains eight major entities:

1. Employee – Stores information about employees involved in software projects.
2. Project – Stores project details and identifies the employee managing each project.
3. Team – Stores teams created for different projects and their team leads.
4. Team_Member – Represents the relationship between employees and teams.
5. Sprint – Stores sprint information associated with projects.
6. Task – Stores tasks associated with sprints and assigned employees.
7. Bug – Stores bugs reported for tasks and the employees responsible for reporting or resolving them.
8. Client – Stores client information associated with projects.

3.2 Database Entities

Entity	Purpose
Employee	Stores employee details
Project	Stores software project details
Team	Stores project teams
Team_Member	Associates employees with teams
Sprint	Stores project sprint information
Task	Stores tasks and their assignments
Bug	Stores reported software bugs
Client	Stores project client information

3.3 Entity Relationships

The major relationships between the entities are:

- An Employee can manage multiple Projects.
- A Project can contain multiple Teams.
- An Employee can lead multiple Teams.
- A Team can contain multiple Employees through Team_Member.
- A Project can contain multiple Sprints.
- A Sprint can contain multiple Tasks.
- An Employee can be assigned multiple Tasks.
- A Task can have multiple Bugs.
- An Employee can report multiple Bugs.
- An Employee can be assigned multiple Bugs for resolution.
- A Project can have multiple Clients.

3.4 Database Design Approach

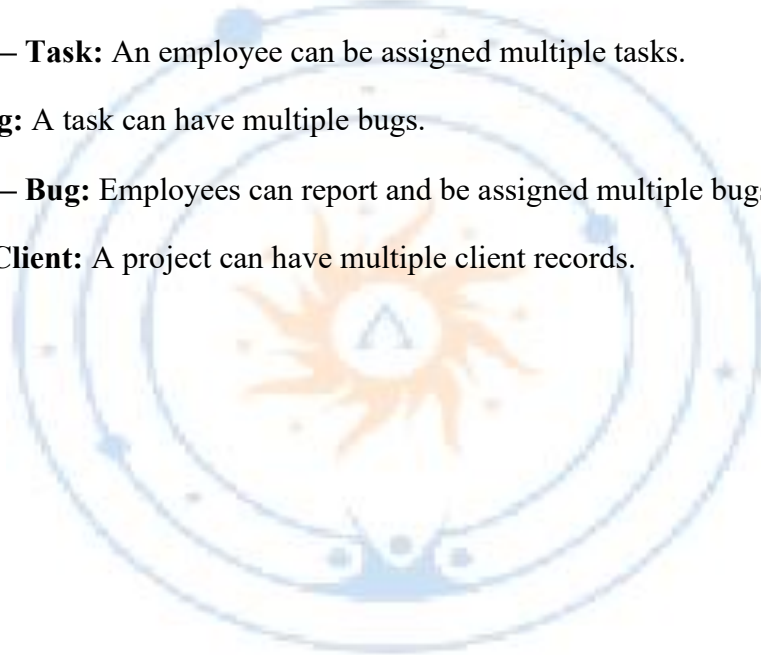
The database is designed using primary keys to uniquely identify records and foreign keys to establish relationships between related tables. Constraints such as NOT NULL, UNIQUE, and CHECK are used where appropriate to maintain data validity and consistency.

The database design provides a structured foundation for SQL-based data management, retrieval, analysis, and reporting.

4. ER DIAGRAM

The relationships represented in the ER diagram include:

1. **Employee — Project:** An employee can manage multiple projects.
2. **Project — Team:** A project can have multiple teams.
3. **Employee — Team:** An employee can lead multiple teams.
4. **Team — Employee:** Teams and employees have a many-to-many relationship implemented through the Team_Member table.
5. **Project — Sprint:** A project can contain multiple sprints.
6. **Sprint — Task:** A sprint can contain multiple tasks.
7. **Employee — Task:** An employee can be assigned multiple tasks.
8. **Task — Bug:** A task can have multiple bugs.
9. **Employee — Bug:** Employees can report and be assigned multiple bugs.
10. **Project — Client:** A project can have multiple client records.



4.1 ER Diagram

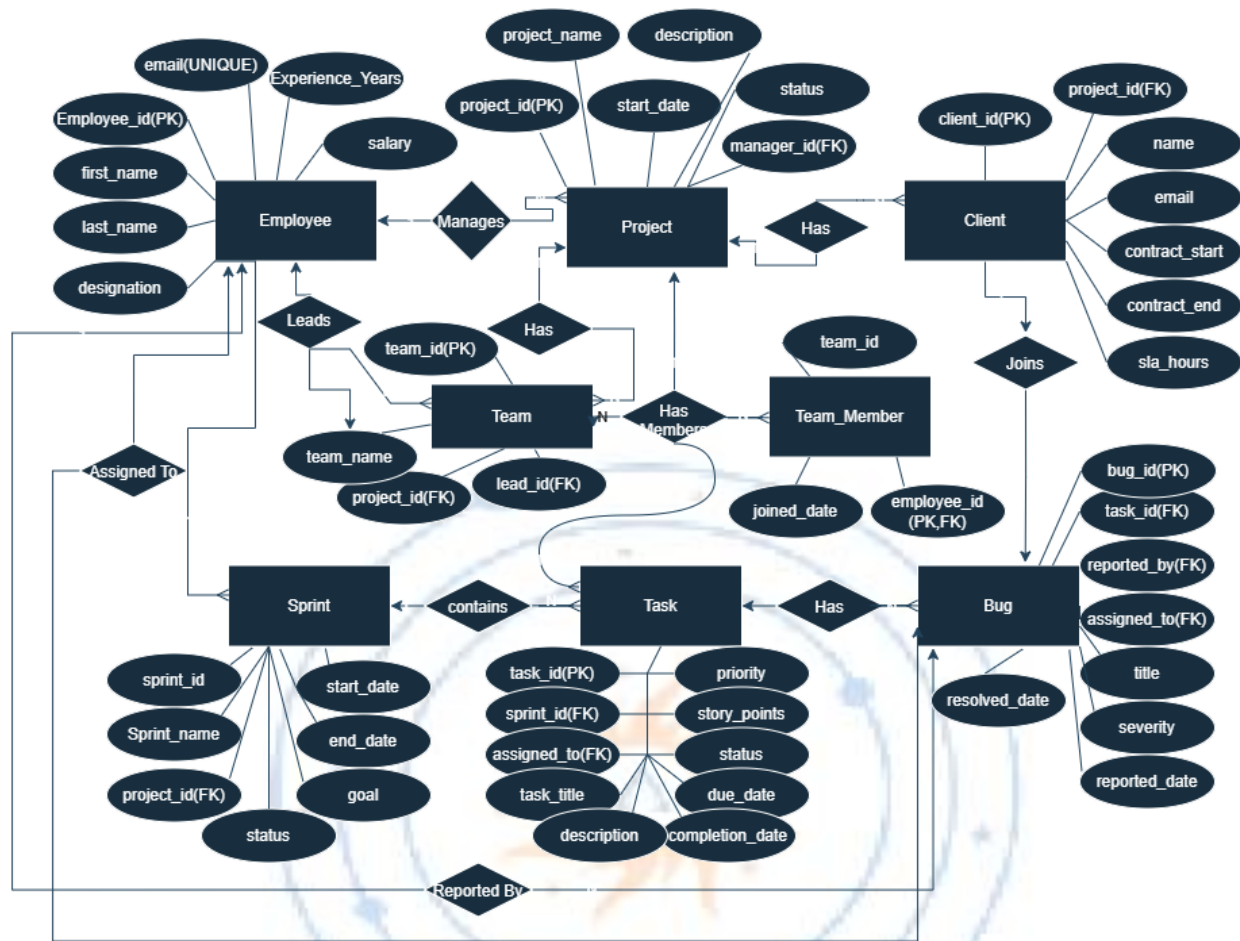


Figure 1: ER Diagram of Software Management Database

4.2 ER Diagram Description

The ER diagram follows the relational database structure implemented in MySQL. Primary keys uniquely identify records within each entity, while foreign keys establish connections between related entities.

The Team_Member entity acts as an associative entity between Team and Employee, allowing multiple employees to belong to multiple teams.

The diagram should correspond directly with the implemented database tables and their primary-key and foreign-key constraints.

5. RELATIONAL SCHEMA

The Software Management Database consists of eight relational tables:

Employee (employee_id, first_name, last_name, designation, email, experience_years, salary)

PK: employee_id

Project (project_id, project_name, description, start_date, end_date, status, manager_id)

PK: project_id | **FK:** manager_id

Team (team_id, team_name, project_id, lead_id)

PK: team_id | **FK:** project_id, lead_id

Team_Member (team_id, employee_id, joined_date)

PK: team_id + employee_id | **FK:** team_id, employee_id

Sprint (sprint_id, sprint_name, project_id, start_date, end_date, goal, status)

PK: sprint_id | **FK:** project_id

Task (task_id, sprint_id, assigned_to, task_title, description, priority, story_points, status, due_date, completion_date)

PK: task_id | **FK:** sprint_id, assigned_to

Bug (bug_id, task_id, reported_by, assigned_to, title, severity, status, reported_date, resolved_date)

PK: bug_id | **FK:** task_id, reported_by, assigned_to

Client (client_id, project_id, name, email, contract_start, contract_end, sla_hours)

PK: client_id | **FK:** project_id

Relationships

Employee → Project: **1:N**

Project → Team: **1:N**

Team ↔ Employee: **M:N** through Team_Member

Project → Sprint: **1:N**

Sprint → Task: **1:N**

Employee → Task: **1:N**

Task → Bug: **1:N**

Project → Client: **1:N**

6. TABLE DESIGN

The database contains eight tables:

Table	Primary Key	Main Foreign Keys
Employee	employee_id	—
Project	project_id	manager_id
Team	team_id	project_id, lead_id
Team_Member	team_id + employee_id	team_id, employee_id
Sprint	sprint_id	project_id
Task	task_id	sprint_id, assigned_to
Bug	bug_id	task_id, reported_by, assigned_to
Client	client_id	project_id

6.1 Employee Table

Attribute	Data Type	Constraint	Description
employee_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique employee identifier
first_name	VARCHAR(50)	NOT NULL	Employee first name
last_name	VARCHAR(50)	NOT NULL	Employee last name
designation	VARCHAR(50)	—	Employee designation
email	VARCHAR(100)	UNIQUE, NOT NULL	Employee email
experience_years	INT	CHECK ≥ 0	Years of experience
salary	DECIMAL(10,2)	—	Employee salary

6.2 Project Table

Attribute	Data Type	Constraint	Description
project_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique project identifier
project_name	VARCHAR(150)	NOT NULL	Project name
description	TEXT	—	Project description
start_date	DATE	—	Project start date
end_date	DATE	—	Project end date
status	VARCHAR(30)	DEFAULT	Current project status
manager_id	INT	FK	Project manager

6.3 Team Table

Attribute	Data Type	Constraint	Description
team_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique team identifier
team_name	VARCHAR(100)	NOT NULL	Team name
project_id	INT	FK, NOT NULL	Associated project
lead_id	INT	FK	Team lead

6.4 Team_Member Table

Attribute	Data Type	Constraint	Description
team_id	INT	PK, FK, NOT NULL	Team identifier
employee_id	INT	PK, FK, NOT NULL	Employee identifier
joined_date	DATE	—	Date employee joined team

6.5 Sprint Table

Attribute	Data Type	Constraint	Description
sprint_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique sprint identifier
sprint_name	VARCHAR(100)	—	Sprint name
project_id	INT	FK, NOT NULL	Associated project
start_date	DATE	—	Sprint start date

end_date	DATE	—	Sprint end date
goal	TEXT	—	Sprint goal
status	VARCHAR(20)	DEFAULT	Sprint status

6.6 Task Table

Attribute	Data Type	Constraint	Description
task_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique task identifier
sprint_id	INT	FK	Associated sprint
assigned_to	INT	FK	Assigned employee
task_title	VARCHAR(200)	NOT NULL	Task title
description	TEXT	—	Task description
priority	VARCHAR(20)	DEFAULT	Task priority
story_points	INT	—	Estimated effort
status	VARCHAR(20)	DEFAULT	Task status
due_date	DATE	—	Task due date
completion_date	DATE	—	Task completion date

6.7 Bug Table

Attribute	Data Type	Constraint	Description
bug_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique bug identifier
task_id	INT	FK	Related task
reported_by	INT	FK, NOT NULL	Employee who reported bug
assigned_to	INT	FK	Employee assigned to bug
title	VARCHAR(200)	NOT NULL	Bug title
severity	VARCHAR(20)	—	Bug severity
status	VARCHAR(20)	DEFAULT	Bug status
reported_date	DATE	DEFAULT	Bug reporting date
resolved_date	DATE	—	Bug resolution date

6.8 Client Table

Attribute	Data Type	Constraint	Description
client_id	INT	PK, NOT NULL, AUTO_INCREMENT	Unique client identifier
project_id	INT	FK, NOT NULL	Associated project
name	VARCHAR(100)	NOT NULL	Client name
email	VARCHAR(100)	NOT NULL	Client email
contract_start	DATE	—	Contract start date
contract_end	DATE	—	Contract end date
sla_hours	INT	—	Service-level agreement hours

6.9 Key Constraints

Constraint	Meaning	Applied To
Primary Key (PK)	Uniquely identifies each record	All tables
Foreign Key (FK)	Establishes relationships between tables	Project, Team, Team_Member, Sprint, Task, Bug, Client
NOT NULL	Prevents empty values in required fields	Employee, Project, Team, etc.
UNIQUE	Prevents duplicate values	Employee.email
CHECK	Ensures values satisfy a condition	Employee.experience_years ≥ 0
DEFAULT	Provides a value automatically when none is supplied	Project.status, Task.status, Bug.status, etc.
AUTO_INCREMENT	Automatically generates numeric IDs	Primary IDs of major tables

Primary and Foreign Key Relationships

- Project.manager_id $\rightarrow$ Employee.employee_id
- Team.project_id $\rightarrow$ Project.project_id
- Team.lead_id $\rightarrow$ Employee.employee_id
- Team_Member.team_id $\rightarrow$ Team.team_id
- Team_Member.employee_id $\rightarrow$ Employee.employee_id
- Sprint.project_id $\rightarrow$ Project.project_id
- Task.sprint_id $\rightarrow$ Sprint.sprint_id
- Task.assigned_to $\rightarrow$ Employee.employee_id

- Bug.task_id $\rightarrow$ Task.task_id
- Bug.reported_by $\rightarrow$ Employee.employee_id
- Bug.assigned_to $\rightarrow$ Employee.employee_id
- Client.project_id $\rightarrow$ Project.project_id



7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The database is normalized from Unnormalized Form (UNF) to Third Normal Form (3NF).

7.1 Unnormalized Form (UNF)

A single software project record might contain **Project_ID**, **Project_Name**, **Manager_Name**, **Team_Name**, **multiple Employee details**, **Sprint details**, **Task details**, and **Bug details** in one record. Repeating employee, task, and bug information can create data redundancy and inconsistency.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains a single atomic value. Employees, teams, sprints, tasks, and bugs are represented as separate records.

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on a composite key are removed. Employee-specific, team-specific, and project-specific attributes are stored in their respective relations. The **Team_Member** table represents the employee–team relationship using a composite key.

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Employee, project, team, sprint, task, bug, and client information are stored in separate relations and connected using foreign keys. This reduces duplication and improves data consistency.

Relation	Reason for Separation
Employee	Stores employee-specific attributes once.
Project	Stores project-specific attributes and manager reference.
Team	Stores team information and project/lead references.
Team_Member	Stores the employee–team relationship.
Sprint	Stores sprint-specific information for projects.
Task	Stores task-specific information and assignments.
Bug	Stores bug-specific information and assignments.
Client	Stores client-specific information linked to projects.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL statements create the tables of the Software Management Database with appropriate primary keys, foreign keys, and constraints..

```
CREATE TABLE Employee (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    designation VARCHAR(50),  
    email VARCHAR(100) UNIQUE NOT NULL,  
    experience_years INT CHECK (experience_years >= 0),  
    salary DECIMAL(10,2)  
);
```

```
CREATE TABLE Project (  
    project_id INT PRIMARY KEY AUTO_INCREMENT,  
    project_name VARCHAR(150) NOT NULL,  
    description TEXT,  
    start_date DATE,  
    end_date DATE,  
    status VARCHAR(30) DEFAULT 'Planning',  
    manager_id INT,  
    FOREIGN KEY (manager_id)  
        REFERENCES Employee(employee_id)  
);
```

```
CREATE TABLE Team (  
    team_id INT PRIMARY KEY AUTO_INCREMENT,  
    team_name VARCHAR(100) NOT NULL,  
    project_id INT NOT NULL,  
    lead_id INT,  
    FOREIGN KEY (project_id)  
        REFERENCES Project(project_id),  
    FOREIGN KEY (lead_id)  
        REFERENCES Employee(employee_id)  
);
```

```

CREATE TABLE Sprint (
    sprint_id INT PRIMARY KEY AUTO_INCREMENT,
    sprint_name VARCHAR(100),
    project_id INT NOT NULL,
    start_date DATE,
    end_date DATE,
    goal TEXT,
    status VARCHAR(20) DEFAULT 'Planned',
    FOREIGN KEY (project_id)
        REFERENCES Project(project_id)
);

```

```

CREATE TABLE Team_Member (
    team_id INT NOT NULL,
    employee_id INT NOT NULL,
    joined_date DATE,
    PRIMARY KEY (team_id, employee_id),
    FOREIGN KEY (team_id)
        REFERENCES Team(team_id),
    FOREIGN KEY (employee_id)
        REFERENCES Employee(employee_id)
);

```

```

CREATE TABLE Task (
    task_id INT PRIMARY KEY AUTO_INCREMENT,
    sprint_id INT,
    assigned_to INT,
    task_title VARCHAR(200) NOT NULL,
    description TEXT,
    priority VARCHAR(20) DEFAULT 'Medium',
    story_points INT,
    status VARCHAR(20) DEFAULT 'Todo',
    due_date DATE,
    completion_date DATE,
    FOREIGN KEY (sprint_id)
        REFERENCES Sprint(sprint_id),

```

```

FOREIGN KEY (assigned_to)
    REFERENCES Employee(employee_id)
);

CREATE TABLE Bug (
    bug_id INT PRIMARY KEY AUTO_INCREMENT,
    task_id INT,
    reported_by INT NOT NULL,
    assigned_to INT,
    title VARCHAR(200) NOT NULL,
    severity VARCHAR(20),
    status VARCHAR(20) DEFAULT 'Open',
    reported_date DATE DEFAULT (CURRENT_DATE),
    resolved_date DATE,
    FOREIGN KEY (task_id)
        REFERENCES Task(task_id),
    FOREIGN KEY (reported_by)
        REFERENCES Employee(employee_id),
    FOREIGN KEY (assigned_to)
        REFERENCES Employee(employee_id)
);

```

```

CREATE TABLE Client (
    client_id INT PRIMARY KEY AUTO_INCREMENT,
    project_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    contract_start DATE,
    contract_end DATE,
    sla_hours INT,
    FOREIGN KEY (project_id)
        REFERENCES Project(project_id)
);

```

8.2 DML – Data Manipulation Language

```

INSERT INTO Employee
(first_name, last_name, designation, email, experience_years, salary)

```

```

VALUES

('Rahul', 'Kumar', 'Project Manager',

'rahul.kumar@example.com', 8, 85000.00);

INSERT INTO Project

(project_name, description, start_date, end_date, status, manager_id)

VALUES

('E-Commerce Platform',

'Development of an online shopping platform',

'2026-01-10', '2026-06-30', 'In Progress', 1);

INSERT INTO Team

(team_name, project_id, lead_id)

VALUES

('Development Team', 1, 1);

INSERT INTO Sprint

(sprint_name, project_id, start_date, end_date, goal, status)

VALUES

('Sprint 1', 1, '2026-01-10', '2026-01-24',

'Initial development', 'Completed');

INSERT INTO Team_Member

(team_id, employee_id, joined_date)

VALUES

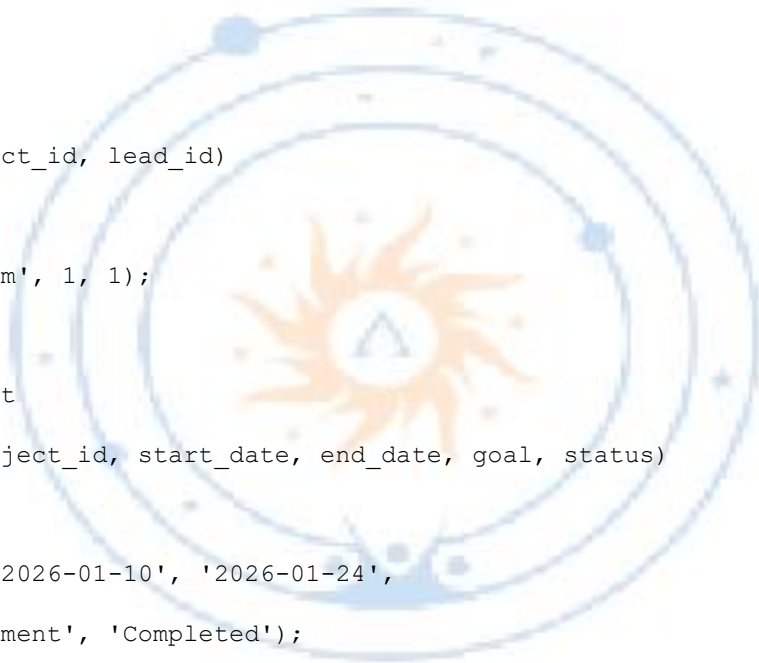
(1, 1, '2026-01-10');

INSERT INTO Task

(sprint_id, assigned_to, task_title, description,

priority, story_points, status, due_date)

```



```
VALUES  
  
(1, 1, 'Database Design',  
    'Design the project database',  
    'High', 5, 'Completed', '2026-01-20');
```

```
INSERT INTO Bug  
(task_id, reported_by, assigned_to, title,  
    severity, status, reported_date)
```

```
VALUES  
  
(1, 1, 1, 'Login Error',  
    'High', 'Open', '2026-01-21');
```

```
INSERT INTO Client  
(project_id, name, email, contract_start, contract_end, sla_hours)
```

```
VALUES  
  
(1, 'ABC Technologies', 'contact@abc.com',  
    '2026-01-10', '2026-12-31', 24);
```

8.3 DQL – Data Query Language

```
SELECT * FROM Employee;
```

```
SELECT project_name, status
```

```
FROM Project
```

```
ORDER BY project_name;
```

```
SELECT t.task_title, e.first_name, e.last_name
```

```
FROM Task t
```

```
JOIN Employee e
```

```
ON t.assigned_to = e.employee_id;
```

```
SELECT p.project_name, e.first_name, e.last_name
```

```
FROM Project p  
JOIN Employee e  
ON p.manager_id = e.employee_id;  
  
SELECT status, COUNT(*) AS total_tasks  
FROM Task  
GROUP BY status;
```



9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display all employees

```
SELECT * FROM Employee;
```

Output:

employee_id	first_name	last_name	designation	email	experience_years	salary
1	Rahul	Kumar	Project Manager	rahul.kumar@company.com	8	85000.00
2	Priya	Sharma	Software Developer	priya.sharma@company.com	4	65000.00
3	Arjun	Reddy	Senior Developer	arjun.reddy@company.com	6	75000.00
4	Sneha	Patel	UI/UX Designer	sneha.patel@company.com	3	55000.00
5	Vikram	Singh	QA Engineer	vikram.singh@company.com	5	60000.00
6	Ananya	Rao	Software Developer	ananya.rao@company.com	2	48000.00
7	Kiran	Reddy	DevOps Engineer	kiran.reddy@company.com	5	70000.00
8	Meera	Nair	Business Analyst	meera.nair@company.com	4	62000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Display all projects

```
SELECT * FROM Project;
```

Output:

project_id	project_name	description	start_date	end_date	status	manager
1	E-Commerce Platform	Online shopping platform with product catalog, ...	2026-01-10	2026-06-30	In Progress	1
2	HR Management System	System for employee registration, attendance a...	2026-02-01	2026-07-31	In Progress	1
3	Mobile Banking App	Mobile banking application with secure authentic...	2026-03-15	2026-09-30	Planning	1
4	AI Analytics Platform	Platform for data analysis, visualization and AI...	2026-04-01	2026-10-31	Planning	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q3. Display all teams

```
SELECT * FROM Team;
```

Output:

team_id	team_name	project_id	lead_id
1	Frontend Team	1	4
2	Backend Team	1	3
3	HR Development Team	2	2
4	Mobile Development Team	3	6
5	Analytics Team	4	7
NULL	NULL	NULL	NULL

Q4. Display all sprints

```
SELECT * FROM Sprint;
```

Output:

sprint_id	sprint_name	project_id	start_date	end_date	goal	status
1	Sprint 1	1	2026-01-10	2026-01-24	Setup project and design database	Completed
2	Sprint 2	1	2026-01-25	2026-02-15	Develop product catalog and authentication	Completed
3	Sprint 3	1	2026-02-16	2026-03-10	Develop shopping cart and payment system	Active
4	Sprint 1	2	2026-02-01	2026-02-20	Employee module development	Completed
5	Sprint 2	2	2026-02-21	2026-03-15	Attendance and leave management	Active
6	Sprint 1	3	2026-03-15	2026-04-05	Mobile application design and authentication	Planned
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q5. Display all tasks

```
SELECT * FROM Task;
```

Output:

task_id	sprint_id	assigned_to	task_title	description	priority	story_points	status	due_date	completion_date
1	1	3	Design Database	Design database structure for the e-commerce ...	High	5	Completed	2026-01-18	2026-01-17
2	2	2	Develop Login Module	Develop secure login and authentication module.	High	8	Completed	2026-02-05	2026-02-04
3	2	6	Product Catalog	Develop product listing and product details func...	Medium	8	Completed	2026-02-10	2026-02-09
4	3	3	Payment Gateway	Integrate payment gateway with the shopping ...	High	13	In Progress	2026-03-05	NULL
5	3	4	Shopping Cart UI	Design and implement shopping cart interface.	Medium	5	In Progress	2026-03-01	NULL
6	4	2	Employee Registration	Develop employee registration module.	High	8	Completed	2026-02-15	2026-02-14
7	5	5	Attendance Module	Develop employee attendance management mo...	Medium	8	In Progress	2026-03-10	NULL
8	6	6	Mobile Login Screen	Create mobile application login interface.	High	5	Todo	2026-03-25	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q6. Display all bugs

```
SELECT * FROM Bug;
```

Output:

bug_id	task_id	reported_by	assigned_to	title	severity	status	reported_date	resolved_date
1	2	5	2	Invalid Login Error	High	Resolved	2026-02-06	2026-02-07
2	3	5	6	Product Image Not Loading	Medium	Open	2026-02-12	NULL
3	4	5	3	Payment Transaction Failure	Critical	Open	2026-02-25	NULL
4	5	5	4	Cart Quantity Issue	Medium	In Progress	2026-02-27	NULL
5	7	5	2	Attendance Calculation Error	High	Open	2026-03-05	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q7. Display all clients

```
SELECT * FROM Client;
```

Output:

client_id	project_id	name	email	contract_start	contract_end	sla_hours
1	1	TechWorld Solutions	contact@techworld.com	2026-01-10	2026-12-31	24
2	2	Global HR Services	contact@globalhr.com	2026-02-01	2026-12-31	12
3	3	FinServe Technologies	contact@finserve.com	2026-03-15	2026-12-31	8
4	4	DataVision Analytics	contact@datavision.com	2026-04-01	2026-12-31	12
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q8. Employees sorted by salary

```
SELECT * FROM Employee

ORDER BY salary DESC;
```

Output:

employee_id	first_name	last_name	designation	email	experience_years	salary
1	Rahul	Kumar	Project Manager	rahul.kumar@company.com	8	85000.00
3	Arjun	Reddy	Senior Developer	arjun.reddy@company.com	6	75000.00
7	Kiran	Reddy	DevOps Engineer	kiran.reddy@company.com	5	70000.00
2	Priya	Sharma	Software Developer	priya.sharma@company.com	4	65000.00
8	Meera	Nair	Business Analyst	meera.nair@company.com	4	62000.00
5	Vikram	Singh	QA Engineer	vikram.singh@company.com	5	60000.00
4	Sneha	Patel	UI/UX Designer	sneha.patel@company.com	3	55000.00
6	Ananya	Rao	Software Developer	ananya.rao@company.com	2	48000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q9. Employees with more than 5 years experience

```
SELECT * FROM Employee

WHERE experience_years > 5;
```

Output:

employee_id	first_name	last_name	designation	email	experience_years	salary
1	Rahul	Kumar	Project Manager	rahul.kumar@company.com	8	85000.00
3	Arjun	Reddy	Senior Developer	arjun.reddy@company.com	6	75000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Projects by status

```
SELECT * FROM Project

WHERE status = 'In Progress';
```

Output:

project_id	project_name	description	start_date	end_date	status	manager
1	E-Commerce Platform	Online shopping platform with product catalog, ...	2026-01-10	2026-06-30	In Progress	1
2	HR Management System	System for employee registration, attendance a...	2026-02-01	2026-07-31	In Progress	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q11. Projects with their managers

```
SELECT p.project_name,
       e.first_name,
       e.last_name
FROM Project p
JOIN Employee e
ON p.manager_id = e.employee_id;
```

Output:

project_name	first_name	last_name
E-Commerce Platform	Rahul	Kumar
HR Management System	Rahul	Kumar
Mobile Banking App	Rahul	Kumar
AI Analytics Platform	Rahul	Kumar

Q12. Teams with their projects

```
SELECT t.team_name,  
       p.project_name  
FROM Team t  
JOIN Project p  
ON t.project_id = p.project_id;
```

Output:

team_name	project_name
Frontend Team	E-Commerce Platform
Backend Team	E-Commerce Platform
HR Development Team	HR Management System
Mobile Development Team	Mobile Banking App
Analytics Team	AI Analytics Platform

Q13. Teams with their leads

```
SELECT t.team_name,  
       e.first_name,  
       e.last_name  
FROM Team t  
JOIN Employee e  
ON t.lead_id = e.employee_id;
```

Output:

team_name	first_name	last_name
Frontend Team	Sneha	Patel
Backend Team	Arjun	Reddy
HR Development Team	Priya	Sharma
Mobile Development Team	Ananya	Rao
Analytics Team	Kiran	Reddy

Q14. Team members with employee names

```

SELECT t.team_name,
       e.first_name,
       e.last_name
FROM Team_Member tm
JOIN Team t ON tm.team_id = t.team_id
JOIN Employee e ON tm.employee_id = e.employee_id;

```

Output:

team_name	first_name	last_name
HR Development Team	Priya	Sharma
Backend Team	Arjun	Reddy
Mobile Development Team	Arjun	Reddy
Frontend Team	Sneha	Patel
HR Development Team	Vikram	Singh
Frontend Team	Ananya	Rao
Mobile Development Team	Ananya	Rao
Backend Team	Kiran	Reddy
Analytics Team	Kiran	Reddy

Q15. Tasks with assigned employee

```

SELECT t.task_title,
       e.first_name,
       e.last_name
FROM Task t
JOIN Employee e
ON t.assigned_to = e.employee_id;

```

Output:

task_title	first_name	last_name
Design Database	Arjun	Reddy
Develop Login Module	Priya	Sharma
Product Catalog	Ananya	Rao
Payment Gateway	Arjun	Reddy
Shopping Cart UI	Sneha	Patel
Employee Registration	Priya	Sharma
Attendance Module	Vikram	Singh
Mobile Login Screen	Ananya	Rao

Q16. Tasks with sprint information

```

SELECT t.task_title,
       s.sprint_name
FROM Task t
JOIN Sprint s

```

```
ON t.sprint_id = s.sprint_id;
```

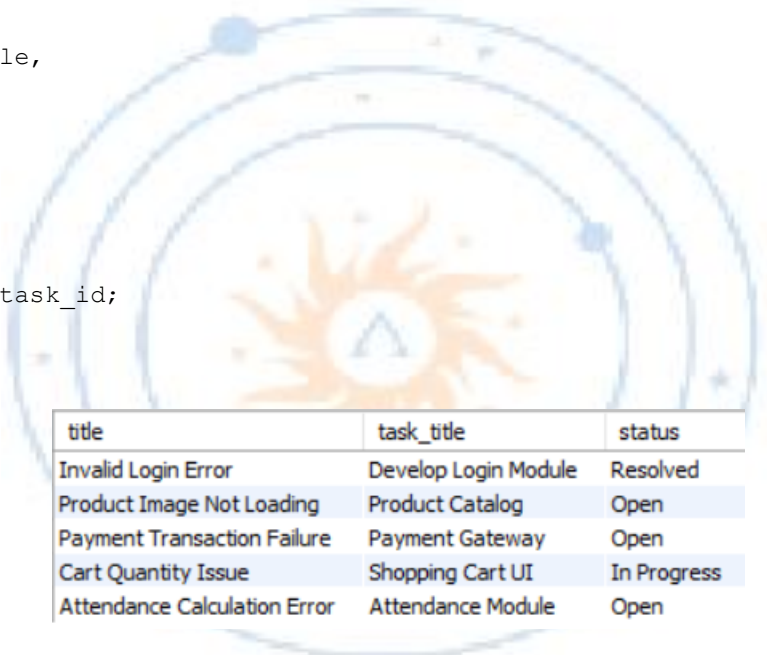
Output:

task_title	sprint_name
Design Database	Sprint 1
Develop Login Module	Sprint 2
Product Catalog	Sprint 2
Payment Gateway	Sprint 3
Shopping Cart UI	Sprint 3
Employee Registration	Sprint 1
Attendance Module	Sprint 2
Mobile Login Screen	Sprint 1

Q17. Bugs with task details

```
SELECT b.title,  
       t.task_title,  
       b.status  
FROM Bug b  
JOIN Task t  
ON b.task_id = t.task_id;
```

Output:



title	task_title	status
Invalid Login Error	Develop Login Module	Resolved
Product Image Not Loading	Product Catalog	Open
Payment Transaction Failure	Payment Gateway	Open
Cart Quantity Issue	Shopping Cart UI	In Progress
Attendance Calculation Error	Attendance Module	Open

Q18. Clients with projects

```
SELECT c.name,  
       p.project_name  
FROM Client c  
JOIN Project p  
ON c.project_id = p.project_id;
```

Output:

name	project_name
TechWorld Solutions	E-Commerce Platform
Global HR Services	HR Management System
FinServe Technologies	Mobile Banking App
DataVision Analytics	AI Analytics Platform

Q19 Count employees

```
SELECT COUNT(*) AS Total_Employees
FROM Employee;
```

Output:

Total_Employees
8

Q20. Average employee salary

```
SELECT AVG(salary) AS Average_Salary
FROM Employee;
```

Output:

Average_Salary
65000.000000

Q21. Highest employee salary

```
SELECT MAX(salary) AS Highest_Salary
FROM Employee;
```

Output:

Highest_Salary
85000.00

Q22. Count tasks by status

```
SELECT status,
COUNT(*) AS Total_Tasks
FROM Task
GROUP BY status;
```

Output:

status	Total_Tasks
Completed	4
In Progress	3
Todo	1

Q23. Count failed results

```
SELECT status,  
       COUNT(*) AS Total_Bugs  
FROM Bug  
GROUP BY status;
```

Output:

status	Total_Bugs
Resolved	1
Open	3
In Progress	1

Q24. Projects managed by each employee

```
SELECT manager_id,  
       COUNT(*) AS Project_Count  
FROM Project  
GROUP BY manager_id;
```

Output:

manager_id	Project_Count
1	4

Q25. Employees earning above average salary

```
SELECT *  
FROM Employee  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM Employee  
);
```

Output:

employee_id	first_name	last_name	designation	email	experience_years	salary
1	Rahul	Kumar	Project Manager	rahul.kumar@company.com	8	85000.00
3	Arjun	Reddy	Senior Developer	arjun.reddy@company.com	6	75000.00
7	Kiran	Reddy	DevOps Engineer	kiran.reddy@company.com	5	70000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q26. Tasks with high priority

```
SELECT *  
FROM Task  
WHERE priority = 'High';
```

Output:

task_id	sprint_id	assigned_to	task_title	description	priority	story_points	status	due_date	completion_date
1	1	3	Design Database	Design database structure for the e-commerce ...	High	5	Completed	2026-01-18	2026-01-17
2	2	2	Develop Login Module	Develop secure login and authentication module.	High	8	Completed	2026-02-05	2026-02-04
4	3	3	Payment Gateway	Integrate payment gateway with the shopping ...	High	13	In Progress	2026-03-05	NULL
6	4	2	Employee Registration	Develop employee registration module.	High	8	Completed	2026-02-15	2026-02-14
8	6	6	Mobile Login Screen	Create mobile application login interface.	High	5	Todo	2026-03-25	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q27. Open bugs

```
SELECT *  
  
FROM Bug  
  
WHERE status = 'Open';
```

Output:

bug_id	task_id	reported_by	assigned_to	title	severity	status	reported_date	resolved_date
2	3	5	6	Product Image Not Loading	Medium	Open	2026-02-12	NULL
3	4	5	3	Payment Transaction Failure	Critical	Open	2026-02-25	NULL
5	7	5	2	Attendance Calculation Error	High	Open	2026-03-05	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q28 Projects having more than one team

```
SELECT project_id,  
       COUNT(*) AS Team_Count  
  
FROM Team  
  
GROUP BY project_id  
  
HAVING COUNT(*) > 1;
```

Output:

project_id	Team_Count
1	2

Q29. Employees whose names start with A

```
SELECT *  
  
FROM Employee  
  
WHERE first_name LIKE 'A%';
```

Output:

employee_id	first_name	last_name	designation	email	experience_years	salary
3	Arjun	Reddy	Senior Developer	arjun.reddy@company.com	6	75000.00
6	Ananya	Rao	Software Developer	ananya.rao@company.com	2	48000.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q30 Task summary with project and employee

```

SELECT p.project_name,
       s.sprint_name,
       t.task_title,
       e.first_name,
       t.status
FROM Project p
JOIN Sprint s ON p.project_id = s.project_id
JOIN Task t ON s.sprint_id = t.sprint_id
JOIN Employee e ON t.assigned_to = e.employee_id;

```

Output:

project_name	sprint_name	task_title	first_name	status
E-Commerce Platform	Sprint 1	Design Database	Arjun	Completed
E-Commerce Platform	Sprint 2	Develop Login Module	Priya	Completed
E-Commerce Platform	Sprint 2	Product Catalog	Ananya	Completed
E-Commerce Platform	Sprint 3	Payment Gateway	Arjun	In Progress
E-Commerce Platform	Sprint 3	Shopping Cart UI	Sneha	In Progress
HR Management System	Sprint 1	Employee Registration	Priya	Completed
HR Management System	Sprint 2	Attendance Module	Vikram	In Progress
Mobile Banking App	Sprint 1	Mobile Login Screen	Ananya	Todo

11. CONCLUSION

The Software Management Database Application was successfully designed and implemented as a relational database system for managing software development information. The database organizes employee, project, team, team member, sprint, task, bug, and client information in separate related tables.

Primary keys, foreign keys, and other constraints were implemented to maintain data integrity and consistency. SQL queries were developed using data retrieval, filtering, sorting, joins, aggregate functions, grouping, HAVING conditions, and subqueries.

The project demonstrates how a relational database can provide centralized and structured management of software project information while reducing data redundancy and maintaining relationships between different entities

12. FUTURE ENHANCEMENTS

The database can be further enhanced by adding:

- Employee time and attendance tracking.
- Project cost and budget management.
- Resource allocation and workload tracking.
- Task and bug priority notifications.
- Advanced project performance reports.
- User authentication and role-based access.
- Automated backup and recovery mechanisms.
- Additional analytical queries and dashboards.

REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.
2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

